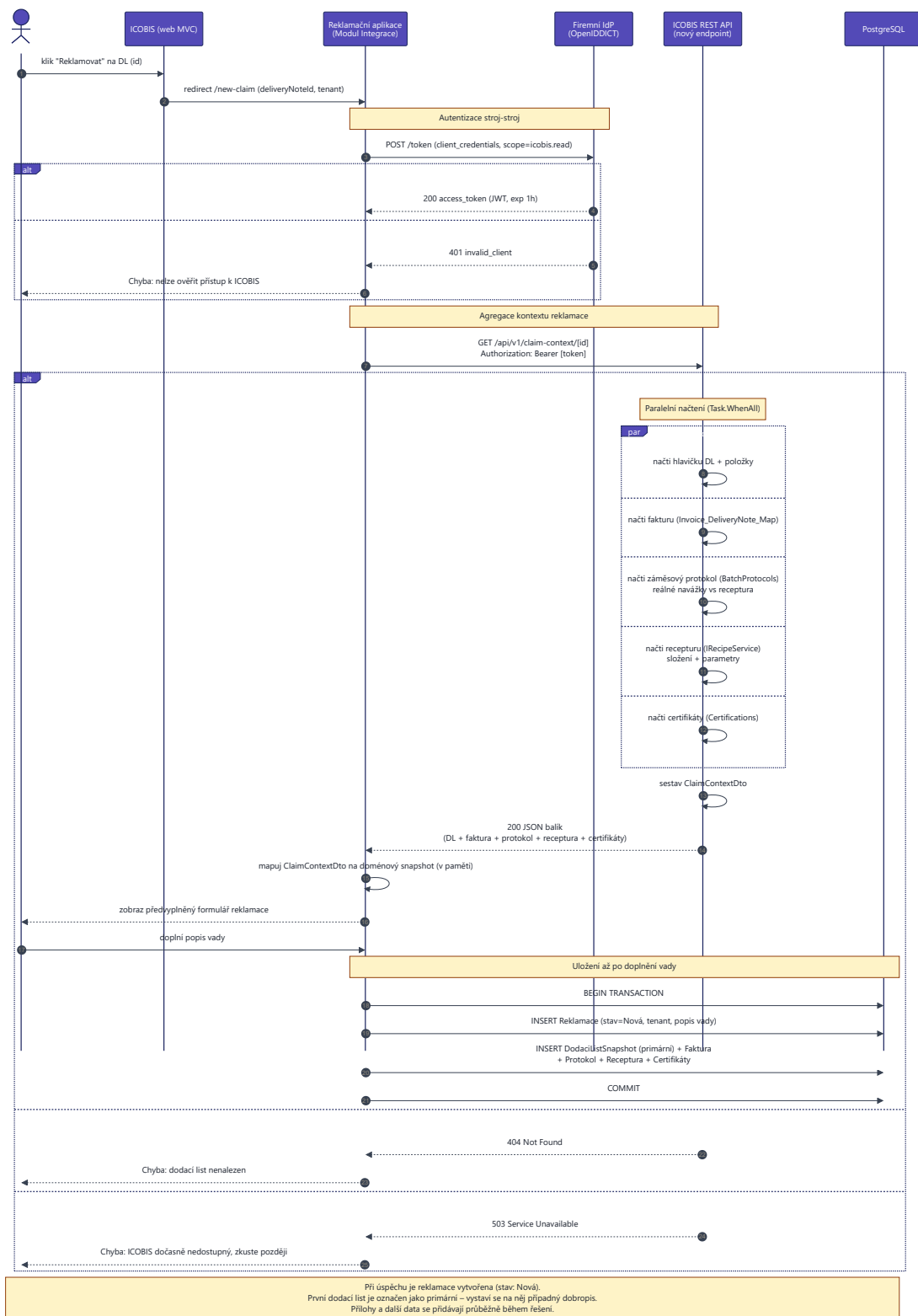


Vytvoření reklamace z ICOBIS

Detailní tok včetně autentizace, agregace výrobních dat a chybových stavů

Kompletní scénář kdy referent v ICOBIS klikne "Reklamovat" na dodacím listu. Diagram ukazuje reálnou složitost: OAuth2 autentizaci stroj-stroj, agregační endpoint `GetClaimContext` který paralelně sbírá data z více vrstev ICOBIS (dodací list, faktura, záměsový protokol s reálnými navážkami, složení receptury, certifikáty), zpracování chybových stavů a uložení imutabilního snapshotu.

Sekvenční diagram



Konkrétní API volání

1. Získání tokenu (OAuth2 client credentials)

POST https://idp.firma.cz/connect/token Content-Type: application/x-www-form-urlencoded grant_type=client_credentials &client_id=reklamace-app &client_secret={secret} &scope=icobis.read icobis.write → 200: { "access_token": "eyJ...", "expires_in": 3600 } → 401: { "error": "invalid_client" }

2. Agregací endpoint

GET https://icobis.firma.cz/api/v1/claim-context/{deliveryNoteId} Authorization: Bearer eyJ... → 200: kompletní kontext (viz struktura níže) → 404: dodací list neexistuje → 401: neplatný/expirovaný token → 503: ICOBIS nedostupný

Struktura odpovědi (ClaimContextDto)

```
{ "deliveryNote": { "id", "documentNo", "dispatched", "customer", "amount" },  
  "invoice": { "number", "amount", "dueDate" } | null, "batchProtocol": {  
    "waterCementRatio", "temperature", "deviations": [...] } | null, "recipe": { "code",  
    "name", "strengthClass", "composition": [...] }, "certificates": [ { "norm", "number"  
  } ] }
```

Kroky toku – vysvětlení

Fáze	Krok	Co se děje a proč
1	Klik Reklamovat	Referent na detailu DL v ICOBIS klikne tlačítko (které jsem do ICOBIS přidal)
2	Redirect	ICOBIS přesměruje do reklamační aplikace s ID dodacího listu a ID tenanta
3	Token	Aplikace získá service token z IdP (client credentials) – stroj-stroj, bez přihlášeného uživatele. Při selhání tok končí chybou.
4	GetClaimContext	Volání agregacího endpointu – jeden request místo pěti samostatných
5	Paralelní agregace	ICOBIS paralelně načte DL, fakturu, záměsový protokol (reálné navážky), recepturu a certifikáty (Task.WhenAll – rychlost)
6	Sestavení DTO	ICOBIS poskládá vše do jednoho JSON balíku a vrátí 200
7	Formulář	Aplikace namapuje data na snapshot (zatím jen v paměti) a zobrazí referentovi předvyplněný formulář

8	Doplnění vady	Referent doplní základní popis vady (přílohy se přidávají průběžně později)
9	Uložení	Až teď se reklamáce uloží v transakci – jeden čistý zápis, imutabilní snapshot
!	Chybové větve	Při 404 (DL neexistuje) nebo 503 (ICOBIS nedostupný) se zobrazí chybová hláška a tok končí – žádná reklamáce nevzniká

Co diagram dokazuje (proti "klikací aplikaci"): Tok zahrnuje OAuth2 autentizaci stroj-stroj, paralelní agregaci dat ze čtyř zdrojů v ICOBIS (včetně záměsového protokolu s reálnými navážkami jako důkazu), kompletní zpracování chybových stavů (404/401/503) a transakční uložení snapshotu. To je netriviální integrační inženýrství.

Proč paralelní načtení nezahrnuje laboratorní pevnosti. Kontrakt s nimi počítá – `ClaimContext` má volitelné pole `labResults`, které systém s integrovanou laboratoří vyplní. Konkrétně ICOBIS je ale nedodá: pevnosti leží v samostatné databázi `KejvalLab`, ke které přistupuje přes podmíněně vytvářená SQL synonyma – u zákazníka bez laboratorního modulu neexistují vůbec a přes exportní vrstvu dostupné nejsou.

Pro ICOBIS integraci proto zůstává pole prázdné a důkazní materiál stojí na **záměsovém protokolu**, který je nativní, dostupný okamžitě a nese skutečné navážky oproti receptuře. Laboratorní protokol vstupuje do reklamáce jako příloha, až zkouška proběhne.